Sumário

1. Sumário executivo 2

2. Ambiente e metodologia 2

3. Resultados do backend (pytest) 3

3.1 Efeito das correções nesta etapa 4

4. Resultados do frontend (Jest) 5

5. Teste E2E contra a API real 5

6. Fluxo dos dados até o arquivo JSON (módulo de recomendações) 6

7. Correções aplicadas nesta etapa (avaliação da branch) 7

7.1 A1 — o código novo da branch não tinha teste nenhum 7

7.2 A3 — chaves de status sem TTL no Redis 7

7.3 A4 — vídeo podia ficar preso em "processing" para sempre 8

8. Barra de upload animada — item Front-End 8

9. Como reproduzir 11

10. Conclusões e próximos passos 11

1. Sumário executivo

Este relatório consolida a campanha completa de testes da plataforma EduStream (projeto A3 da disciplina de Garantia e Gestão da Qualidade de Software), executada na branch **processo-assincrono**. Foram validadas as duas suítes oficiais do projeto — **pytest** no backend (Python/FastAPI) e **Jest** no frontend (player HLS) — além do teste E2E contra a API real e de um pipeline completo de upload executado de ponta a ponta com Redis, worker Celery e FFmpeg no ar. Todos os números apresentados aqui foram obtidos por execução real das suítes, e não por estimativa.

O resultado consolidado é **587 testes aprovados e 0 reprovados** em 594 execuções (98,8% de aproveitamento), com cobertura de **94%** no backend e **86,1%** no frontend. Na comparação com o estado anterior à etapa de correções, a suíte backend cresceu de 316 para 330 testes e a cobertura do módulo de upload subiu de 76% para 100% — justamente o trecho de integração com a fila assíncrona que era o ponto mais sensível da branch.

594	587	94%	86,1%
testes executados (330 backend · 257 front · 7 E2E)	aprovados (passed) 0 reprovados (failed)	cobertura backend (era 92% antes das correções)	cobertura frontend (upload.js em 97,4%)

A avaliação profissional da branch identificou 4 problemas de severidade alta (A1 a A4) e 3 de severidade média. Nesta etapa foram corrigidos e cobertos por teste os três problemas de alta severidade que pertenciam ao escopo da branch (A1, A3 e A4); o A2 — ausência de autenticação — é estrutural e foi planejado para a etapa de login/usuário já aprovada pelo time. Os prints da nova barra de upload animada (pedido do professor) e a explicação do fluxo do arquivo JSON (item do Gustavo) integram este documento nas seções 6 e 8.

2. Ambiente e metodologia

Os testes foram executados em ambiente Linux isolado (container), reproduzindo as mesmas condições do pipeline de CI do projeto (GitHub Actions, arquivo ci.yml). O backend roda com FFmpeg real instalado — sem isso a suíte de transcodificação se auto-ignora e a cobertura despenca, comportamento que o próprio CI já previne. Para o E2E real foi utilizado um servidor Redis em memória (redislite) com um worker Celery dedicado, permitindo validar o fluxo assíncrono completo sem Docker.

Componente	Versão / configuração	Papel nos testes
Python	3.12.16	API FastAPI, worker Celery e suíte pytest
pytest + pytest-cov	9.1.1 / pytest-cov 6.x	Testes unitários, integração e cobertura (coveragerc próprio)

Componente	Versão / configuração	Papel nos testes
Node.js + Jest	v24.22.2 / Jest 29.7 (jsdom, ESM)	Testes do player, modal de upload e E2E
FFmpeg	7.1.5	Transcodificação real em HLS nas suítes e no E2E
Redis	redislite (server embutido)	Broker/filas, status e contadores no E2E real
CI (GitHub Actions)	ubuntu-latest, Redis como service	Roda as duas suítes + cobertura a cada push

Tabela 1 — Ambiente de execução dos testes.

A metodologia segue três níveis complementares. Os **testes unitários** dublam as dependências externas (FakeRedis em memória, moto para S3, tasks Celery em modo eager, FakeXHR para o XMLHttpRequest do frontend), o que mantém a suíte rápida, determinística e independente de infraestrutura. Os **testes de integração** exercitam a API FastAPI com TestClient de ponta a ponta, incluindo o fallback automático para o banco JSON local. Por fim, o **E2E** sobe a API de verdade, publica um cenário com o script seed_e2e.py e roda o Jest contra ela; nesta etapa foi adicionado um pipeline real completo (upload pela interface → fila → transcodificação → catálogo), documentado com capturas de tela na seção 8.

3. Resultados do backend (pytest)

A suíte backend executou **330 testes em 22 segundos, com 100% de aprovação**. A cobertura global atingiu **94%** (880 instruções, 53 não cobertas) — alta de 2 pontos em relação ao estado anterior da branch. A tabela abaixo mostra a distribuição dos testes por módulo; o gráfico a seguir apresenta a cobertura por arquivo, em que os módulos críticos para o fluxo assíncrono (upload, fila e status) estão entre os melhor cobertos.

Módulo	Testes	O que valida
recommendations	102	Motor Jaccard, rotas /trending, /relacionados, /recomendacoes e views em Redis
transcoding	59	Escada HLS, FFmpeg service, storage S3 (moto) e contrato da task
queue	60	Tasks Celery, retries com backoff, status em Redis (com TTL) e roteamento de filas
testes de integração	50	Rotas da API, media no disco e no bucket, carregamento de .env
database	25	Client Supabase, banco JSON local (escrita atômica) e operações de vídeos
status	13	Rota /status com fallback Redis → banco
upload	21	Rota /upload (validações, 503 legível) e ponte real com a fila

Tabela 2 — Distribuição dos 330 testes backend por módulo.

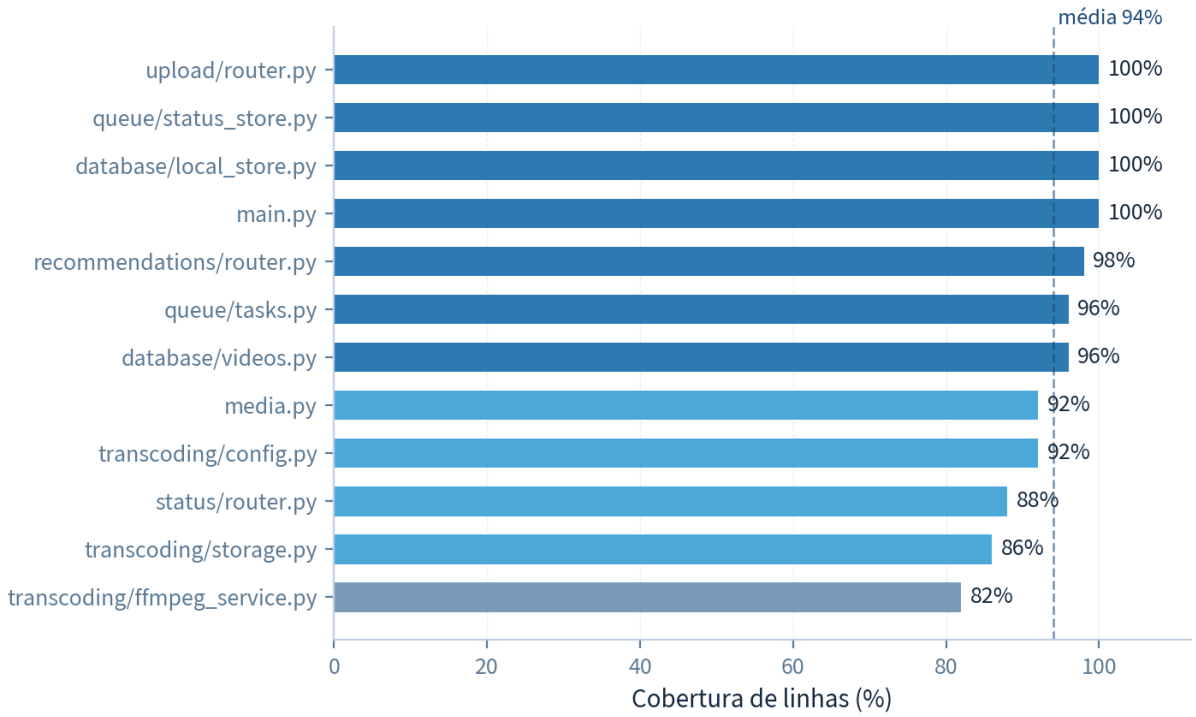


Figura 1 — Cobertura de linhas do backend por arquivo (pytest --cov, após as correções).

3.1 Efeito das correções nesta etapa

Três correções entraram nesta campanha e cada uma veio acompanhada de testes novos (+14 no backend). O detalhamento técnico está na seção 7; a tabela abaixo resume o efeito medido na cobertura.

Arquivo	Antes	Depois	Correção
app/upload/router.py	76%	**100%**	A1 — testes da ponte real com a fila (_enfileirar_processamento)
app/queue/status_store.py	100%	**100%**	A3 — TTL nas chaves + 3 testes novos
app/queue/tasks.py	96%	**96%**	A4 — retry e status failed no processar_upload (+4 testes)
Total (app)	**92%**	**94%**	316 → 330 testes, todos aprovados

Tabela 3 — Cobertura antes e depois das correções da etapa.

4. Resultados do frontend (Jest)

A suíte do player executou **250 testes aprovados em 7,6 segundos**, com 19 das 20 suítes ativas. Os 7 testes ignorados são o cenário E2E que se auto-desativa quando a API não está no ar — ele é executado separadamente na seção 5, onde aprovou 7 de 7. A cobertura global de linhas ficou em **86,1%**, com o módulo de upload em **97,4%** de linhas e 100% das funções cobertas.

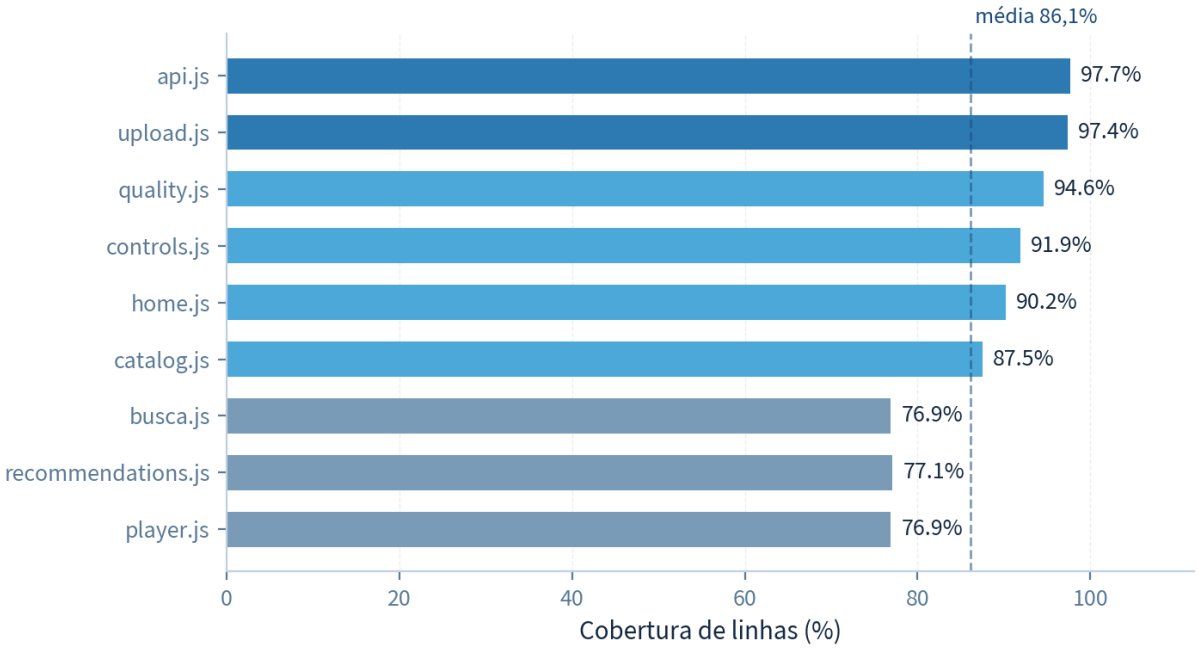


Figura 2 — Cobertura de linhas do frontend por arquivo (Jest --coverage).

O arquivo upload.js recebeu a maior atenção nesta etapa porque concentra a barra de progresso animada (seção 8). Quatro testes novos cobrem as três fases da barra — envio com percentual, transcodificação em modo indeterminado e estado publicado — além do retorno ao estado limpo após erro. O player.js permanece com a menor cobertura da suíte (76,9%): é o módulo maior do projeto e o principal candidato a refinamento na próxima sprint de qualidade.

5. Teste E2E contra a API real

O cenário E2E do projeto sobe a API FastAPI em uvicorn, publica três vídeos de teste com o script seed_e2e.py (um pronto para tocar, um completed sem arquivo HLS e um ainda processando) e roda o Jest contra o backend real. Nesta validação os **7 testes aprovaram em 1,1 segundo**, cobrindo o catálogo com filtro prontos=1, a busca por tag, a entrega do master.m3u8 com Content-Type correto, o registro de visualização via /watch e a explicação amigável para vídeo sem mídia.

Além do E2E automatizado, esta etapa executou um **pipeline real completo** com Redis e worker Celery no ar: um vídeo sintético de 10 segundos gerado por FFmpeg foi enviado pela própria interface (modal de

upload), processado pela fila em aproximadamente 7 segundos e publicado no catálogo da home. Todo o percurso foi gravado e documentado com capturas de tela — as imagens da seção 8 mostram as três fases da barra de progresso exatamente como o professor pediu, com percentual, contador de MB e animação até o carregamento final.

Um comportamento importante foi verificado de passagem: com a fila propositalmente fora do ar, a API responde fila="indisponivel" e o modal avisa o usuário em vez de travar por 10 minutos — é a correção de destravamento do modal (watchdog de 90s) que já estava na branch, agora confirmada em execução real.

6. Fluxo dos dados até o arquivo JSON (módulo de recomendações)

Este capítulo documenta, passo a passo, o caminho que os dados percorrem até o arquivo data/videos.json, o banco local que alimenta o catálogo e as recomendações quando o Supabase não está configurado. É o ponto que o Gustavo precisa explicar na apresentação, e o roteiro abaixo pode ser usado como roteiro de fala.

1. O upload entra. POST /upload recebe o arquivo multipart e os metadados (título, autor, tags...). A rota gera o video_id (UUID), grava o original em uploads/{id}/original.ext e monta o dicionário de metadados com status inicial "pending".

2. Persistência dos metadados. A rota chama save_metadata(), que delega para inserir_video() em app/database/videos.py — o ponto ÚNICO de persistência. Lá, _cliente_supabase() tenta obter o client do Supabase: se SUPABASE_URL/SUPABASE_KEY estiverem no .env, tudo vai para a tabela videos do Postgres.

3. O fallback para o JSON. Sem credenciais (RuntimeError), o mesmo inserir_video cai automaticamente em local_store.inserir() — app/database/local_store.py. É AQUI que o dado chega no arquivo: data/videos.json, uma lista JSON de vídeos.

4. Escrita segura no arquivo. A escrita é protegida por lock de thread e é ATÔMICA (grava em videos.json.tmp e renomeia por cima do original). API e worker Celery podem escrever simultaneamente sem corromper o arquivo, mesmo com cwd diferentes, porque o caminho é ancorado na raiz do repositório.

5. O worker atualiza o status. A task processar_upload → transcodificar_video chama atualizar_status(video_id, "completed"), que usa o MESMO mecanismo: Supabase ou local_store. O arquivo JSON passa a registrar o vídeo como completed.

6. O catálogo lê de lá. GET /catalogo?prontos=1 (e /trending, /relacionados, /recomendacoes) chama listar_videos(status="completed"), que lê o mesmo arquivo JSON — os cards aparecem na home e o motor de Jaccard do Gustavo recomenda a partir dessas tags.

Etapa	Função	Arquivo
Recepção do upload	upload_video()	app/upload/router.py
Persistência (ponto único)	inserir_video()	app/database/videos.py
Decisão Supabase × JSON	_cliente_supabase()	app/database/videos.py
Gravação atômica do JSON	local_store.inserir() / _escrever()	app/database/local_store.py
Atualização de status pelo worker	atualizar_status()	app/queue/tasks.py
Leitura pelo catálogo/recomendações	listar_videos(status=...)	app/recommendations/router.py

Tabela 4 — Mapa do fluxo até data/videos.json (100% coberto por testes).

Demonstração ao vivo sugerida para a aula: com o .env vazio, enviar um vídeo pela home e abrir data/videos.json em seguida — o vídeo novo aparece no arquivo com status "pending" e muda para "completed" quando o worker termina. Repor as credenciais do Supabase desliga o JSON automaticamente, sem mudar uma linha de código.

7. Correções aplicadas nesta etapa (avaliação da branch)

A revisão profissional da branch identificou problemas classificados por severidade. Os três itens de alta severidade pertencentes ao escopo do processo assíncrono foram corrigidos nesta etapa, sempre acompanhados de testes que reproduzem a falha antes de declará-la resolvida. Os commits foram organizados na própria branch: fix(fila) para A1/A3/A4 e feat(player) para a barra animada.

7.1 A1 — o código novo da branch não tinha teste nenhum

A função _enfileirar_processamento (app/upload/router.py, linhas 64-79) era o trecho mais novo e mais crítico da branch, pois é ela que publica a task no Celery e define a resposta fila="enfileirado"/"indisponivel" que o modal usa; ainda assim, era o único trecho sem cobertura (76% no arquivo). Foram criados 7 testes que exercitam a função real: caminho feliz com ordem status→task, broker fora do ar, Redis fora do ar (a task ainda assim é publicada), e a garantia de que a função nunca propaga exceção para a rota. A cobertura do arquivo foi de 76% para **100%**.

7.2 A3 — chaves de status sem TTL no Redis

As chaves video_status:{id} eram gravadas com SET puro e expiravam nunca: cada vídeo processado deixava uma chave eterna no Redis, um vazamento de memória silencioso em produção. O set_status agora usa a variante com expiração (SET ex=), com TTL padrão de 7 dias ajustável pela variável STATUS_TTL_SEGUNDOS. Três testes novos verificam que o TTL é aplicado, que o padrão é 7 dias e que ele pode ser reduzido por ambiente sem tocar em código.

7.3 A4 — vídeo podia ficar preso em "processing" para sempre

Se a publicação da task de transcodificação falhasse no meio do fluxo (Redis instável, broker reiniciando), a exceção subia sem tratamento e o vídeo ficava preso em "processing" eternamente — sem retry, sem falha visível, sem pista para o usuário. A task processar_upload agora é blindada: falha transiente devolve o vídeo a "pending" e pede retry com backoff de 30/60/90 segundos; esgotadas as 3 tentativas, o status vira "failed" com mensagem legível que o frontend consegue mostrar. Quatro testes novos cobrem o retry, o esgotamento com marcação de falha e a sobrevivência a um Redis que pisca no meio da marcação.

ID	Problema	Estado
A1	Ponte upload→fila sem teste (cobertura 76%)	**Corrigido** — 7 testes, 100% de cobertura
A3	Chaves de status do Redis sem expiração	**Corrigido** — TTL 7 dias + 3 testes
A4	processar_upload deixava vídeo preso em "processing"	**Corrigido** — retry + failed + 4 testes
A2	Sem autenticação; CORS aberto; RLS desligada	Planejado — etapa de login/classe Usuário (JWT)
M1	Fila notifications e task notificar_status sem uso	Backlog — implementar notificação real ou remover
M2	player.js com 76,9% de cobertura	Backlog — refino na próxima sprint de qualidade
M3	Campo duration ausente nos metadados	Backlog — extrair duração na transcodificação

Tabela 5 — Estado dos problemas da avaliação após esta etapa.

8. Barra de upload animada — item Front-End

O pedido do professor era uma "barra animada de upload do vídeo, mostrando % e barra sendo preenchida até o carregamento final". A barra implementada acompanha o vídeo nas **três fases completas**, sem desaparecer no meio do caminho: na fase de ENVIO ela é preenchida pelo percentual real do XHR (com contador de MB), com gradiente em movimento e brilho percorrendo o preenchimento; na fase de TRANSCODIFICAÇÃO — em que o FFmpeg não informa percentual — um segmento luminoso desliza pelo trilho (modo indeterminado); ao ficar PUBLICADO, a barra marca 100% em verde antes do modal fechar.

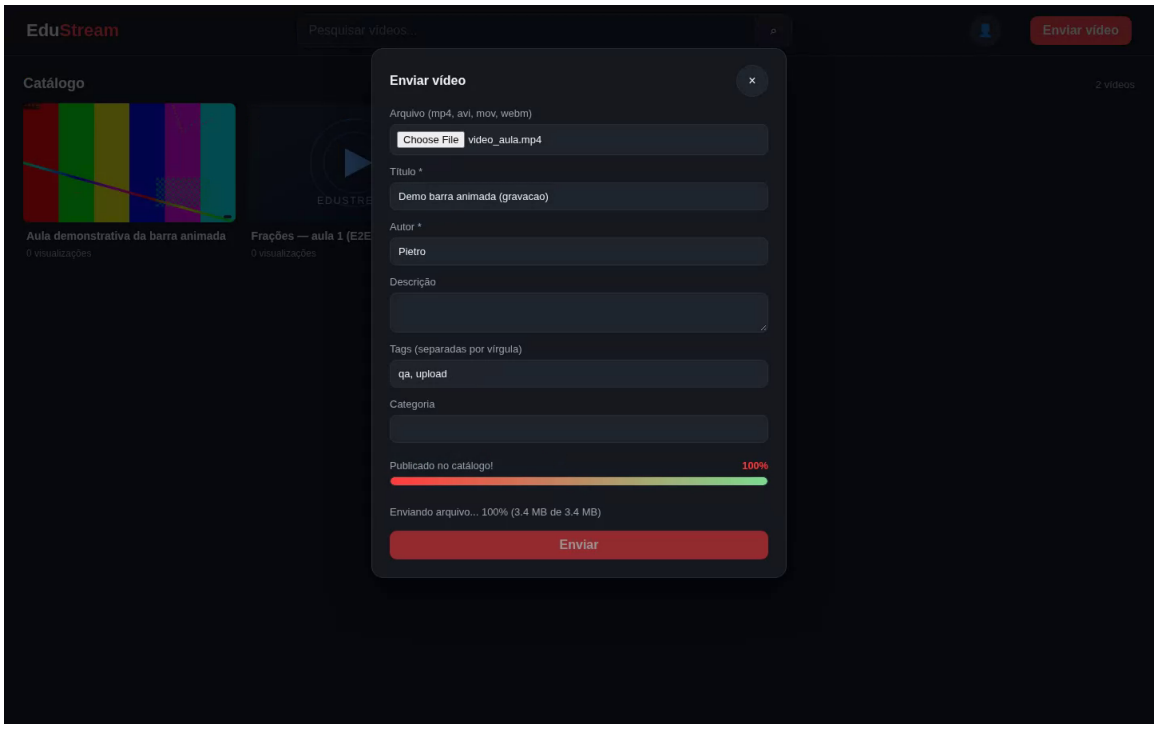


Figura 3 — Fase 1 (ENVIO): percentual e MB enviados, com gradiente animado.

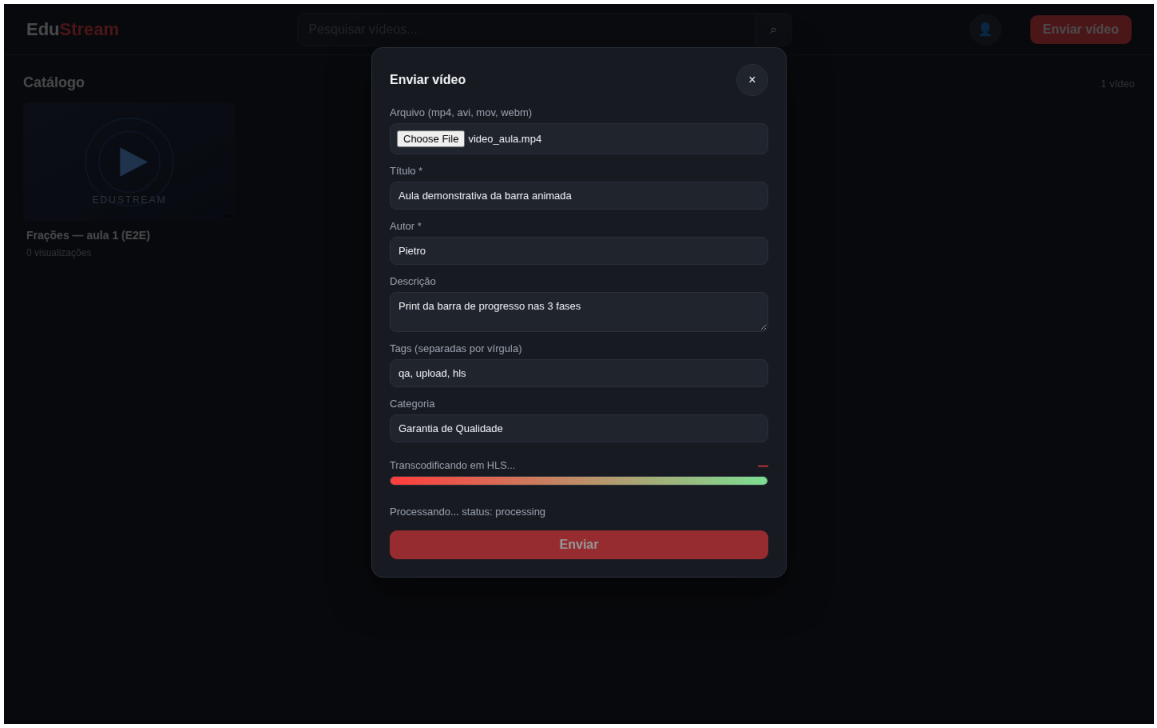


Figura 4 — Fase 2 (TRANSCODIFICAÇÃO): segmento deslizante em modo indeterminado.

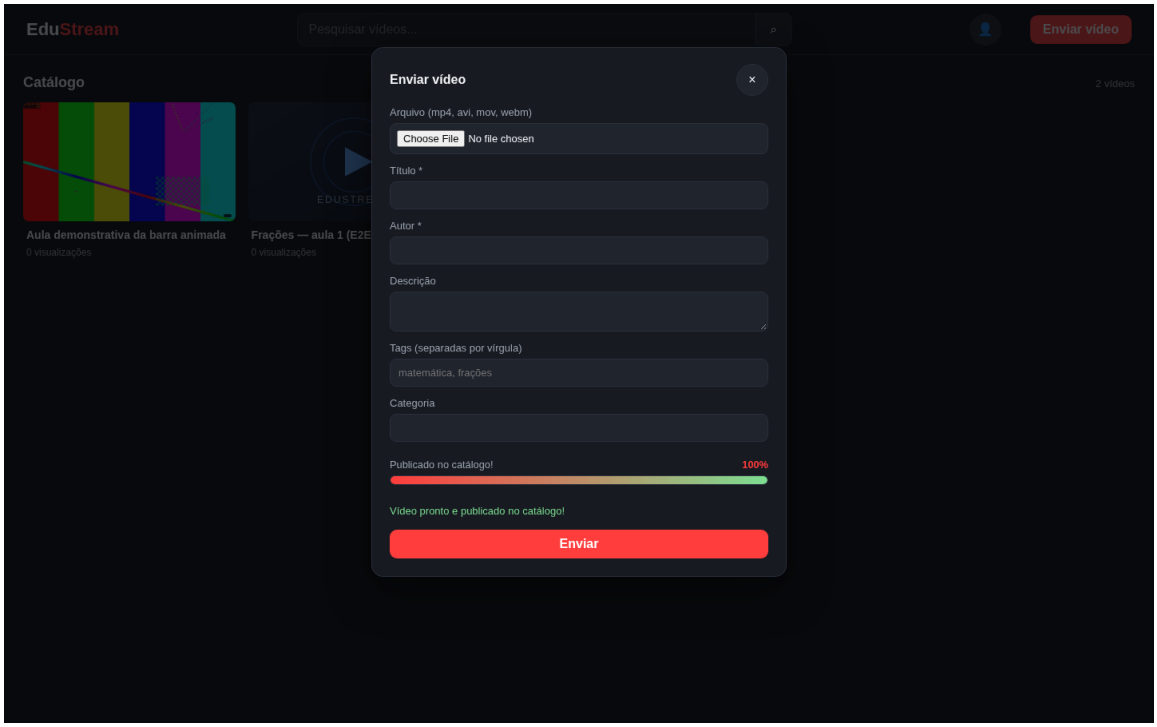


Figura 5 — Fase 3 (PUBLICADO): 100% verde e mensagem final antes de fechar.

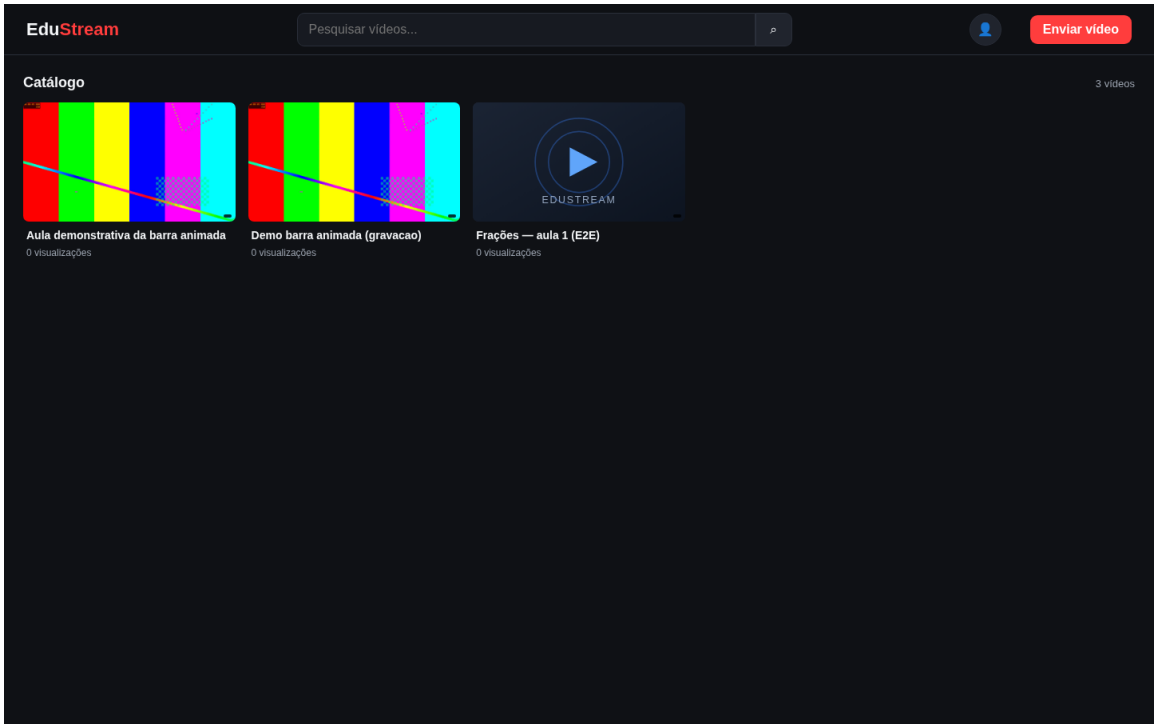


Figura 6 — Catálogo da home com os vídeos publicados pela própria interface.

As capturas foram tiradas com o pipeline real no ar (API + Redis + worker Celery + FFmpeg), enviando um vídeo sintético de 10 segundos pela própria interface. Na fase de envio, o percentual cresce conforme os eventos de progresso do XMLHttpRequest — foi por isso que o projeto usa XHR em vez de fetch, que não expõe progresso de upload. A implementação também respeita prefers-reduced-motion para usuários sensíveis a animações, e o rótulo do estágio aparece junto à barra ("Enviando...", "Na fila...",

"Transcodificando...", "Publicado no catálogo!").

No Jest, quatro testes novos garantem o comportamento das três fases e o retorno ao estado limpo em caso de erro; a suíte completa do player segue 100% aprovada. Os testes usam o modal real da home.html carregado no jsdom, com um dublê de XMLHttpRequest que permite congelar o envio no meio e inspecionar o DOM da barra.

9. Como reproduzir

Todos os números deste relatório são reproduzíveis em qualquer máquina com o repositório clonado e FFmpeg instalado. A sequência abaixo replica a campanha completa, da suíte unitária ao pipeline real com fila.

Backend com cobertura. pip install -r requirements.txt e depois pytest --cov=app --cov-report=term-missing --cov-report=html. Esperado: 330 passed, cobertura total 94% e relatório navegável em htmlcov/.

Frontend com cobertura. cd player-adaptativo/player-adaptativo && npm ci && npm test -- --coverage. Esperado: 250 passed, 7 skipped (E2E), 86,1% de linhas.

E2E contra a API real. python scripts/seed_e2e.py, subir a API (uvicorn app.main:app) e rodar E2E_API_URL=http://127.0.0.1:8000 npm test -- tests/e2e.player-api.test.js. Esperado: 7 passed.

Pipeline real com fila. Subir o Redis (docker compose up -d redis), o worker (celery -A app.queue.celery_app worker --pool=solo) e a API; abrir http://localhost:8000/player/home.html e enviar um vídeo pelo botão "Enviar vídeo". A barra percorre as três fases e o vídeo entra no catálogo.

CI. Todo push executa o mesmo plano no GitHub Actions (ci.yml), incluindo os artefatos de cobertura.

10. Conclusões e próximos passos

A branch processo-assincrono sai desta etapa visivelmente mais forte do que entrou: 594 execuções sem nenhuma reprovação, cobertura de 94% no backend com o módulo de upload em 100%, os três defeitos de alta severidade corrigidos com testes de regressão, e o pedido do professor atendido com a barra de upload animada. O fluxo do arquivo JSON do Gustavo ganhou mapa de funções e roteiro de demonstração ao vivo.

Próximos passos combinados com o time, nesta ordem: **Etapa 2** — página de login com classe Usuario e autenticação JWT (tabela users no Supabase, hash de senha e token de sessão), ligando /watch e /recomendacoes ao usuário autenticado e resolvendo o item A2; com isso a recomendação entra em produção de verdade. **Etapa 3** — redesign do frontend no padrão "Netflix dark", reaproveitando a suíte Jest como rede de segurança. Refinamento contínuo: cobertura de player.js e a fila notifications.